

Class 4A

Subject ADA LAB

Sl No.	Date	Title	Page No.	Teacher Sign/Remarks
1	23/2/26	Binary search & linear search.	10	23/2
2	2/3/26	Leetcode.	10	2/3/26
3	9/3/26	Merge sort	10	9/3
4	16/3/26	Quick sort.	10	16/3
5	30/3/26	Prim's & Kruskal.	5+5=10	30/3
6	13/4/26	Topological. & Leetcode 207	10	13/4
7	21/4/26	Johnson - Trotter & Heap sort (bottom-up approach).	10	21/4
8	4/5/26	Floyd's algorithm & Leetcode 1334.	10	4/5
9	11/5/26	Dijkstra.	10	11/5
10	11/5/26	0/1 knapsack & leetcode - 494.	10	11/5
11	11/5/26	Fractional knapsack & leetcode - 316	10	11/5
12	25/5/26	N-queen's problem.	10	25/5

Linear search.

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[6];
```

```
    arr = {1, 4, 5, 6, 7, 8};
```

```
    int key = 5;
```

```
    int found = 0;
```

```
    for (int i = 0; i < 6; i++) {
```

```
        if (arr[i] == key)
```

```
            found = 1;
```

```
    }
```

```
    if (found == 1) {
```

```
        printf("key found");
```

```
    }
```

```
    else
```

```
        printf("key is not present");
```

```
}
```

binary search algorithm

```
int binarysearch (int arr[], int low, int high, int target) {
```

```
    if (low > high)
```

```
        return -1;
```

```
    int mid = low + (high - low) / 2;
```

```
    if (arr[mid] == target)
```

```
        return mid;
```

```
    else if (arr[mid] > target)
```

```
        return (arr, low, mid - 1, target);
```

```
    else
```

```
        return (arr, mid + 1, high, target);
```

```
}
```

leetcode 3867

```
void computeTotient (int n) {
```

```
    for (int i=1; i<=n; i++)
```

```
        phi[i] = i;
```

```
    for (int i=2; i<=n; i++) {
```

```
        if (phi[i] == i) {
```

```
            for (int j=i; j<=n; j+=i) {
```

```
                phi[j] -= phi[j]/i;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
long long gcdsum (int n) {
```

```
    computeTotient(n);
```

```
    long long res = 0;
```

```
    for (int i=1; i<=n; i++) {
```

```
        for (int j=2; i*j<=n; j++) {
```

```
            result += (long long) i * phi[j];
```

```
        }
```

```
    }
```

```
    return res;
```

```
}
```

leetcode 3658

```
int gcdOfOddEvenSums (int n) {
```

```
    return n;
```

```
}
```

sum of even. n $\Rightarrow 2 + 4 + 6 + \dots$

$2(1 + 2 + 3 + \dots)$

$2 \frac{n(n+1)}{2} = n(n+1)$

sum of odd = $1 + 3 + 5 + \dots$

$= n^2$

$\therefore \text{gcd}(n^2, n(n+1))$

$\text{ngcd}(n, n+1) = n \times 1 = n$

leetcode 2447

```
int gcd (int a, int b) {
```

```
    while (b != 0) {
```

```
        int t = b;
```

```
        b = a % b;
```

```
        a = t;
```

```
    }
```

```
    return a;
```

```
}
```

```
int subarrayGCD (int* nums, int numsSize, int k) {
```

```
    int count = 0;
```

```
    for (int i = 0; i < numsSize; i++) {
```

```
        int g = 0;
```

```
        for (int j = i; j < numsSize; j++) {
```

```
            g = gcd(g, nums[j]);
```

```
            if (g == k) {
```

```
                count++;
```

```
            }
```

```
            if (g < k) {
```

```
                break;
```

```
        }
```

```
    }
```

```
}
```

```
    return count;
```

```
}
```

leetcode 3336.

```
long long dp[205][205][205];  
long long solve (int i, int g1, int g2) {  
    if (i == n) {  
        if (g1 == g2 && g1 != 0)  
            return 1;  
        return 0;  
    }  
}
```

```
if (dp[i][g1][g2] != -1)  
    return dp[i][g1][g2];
```

```
long long ans = 0;  
ans = (ans + solve(i+1, g1, g2)) % MOD;
```

```
int newg1 = (g1 == 0) ? nums[i] : gcd(g1, nums[i]);  
ans = (ans + solve(i+1, newg1, g2)) % MOD;
```

```
int newg2 = (g2 == 0) ? nums[i] : gcd(g2, nums[i]);  
ans = (ans + solve(i+1, g1, newg2)) % MOD;
```

```
return dp[i][g1][g2] = ans;
```

}

```
int subsequencePairCount (int * arr, int size) {
```

```
    n = size;
```

```
    for (int i = 0; i < n; i++) {
```

```
        nums[i] = arr[i];
```

```
    }
```

```
    memset(dp, -1, sizeof(dp));
```

```
    return solve(0, 0, 0);
```

}

Merge sort.

Algorithm.

Merge-sort (A , low, high) d.

if (low < high) d.

mid = (low + high) / 2.

Merge-sort (A , low, mid)

Merge-sort (A , mid + 1, high)

Merge (A , low, mid, high)

}

}

Merge (A , low, mid, high) d.

i = low, j = mid + 1, k = low;

while (i ≤ mid && j ≤ high) d.

if ($A[i] \leq A[j]$) d.

$B[k++] = A[i++]$.

}

else {

$B[k++] = A[j++]$;

}

while (i ≤ mid)

$B[k++] = A[i++]$.

while (j ≤ high)

$B[k++] = A[j++]$

for (i = low → high)

$A[i] = B[i]$

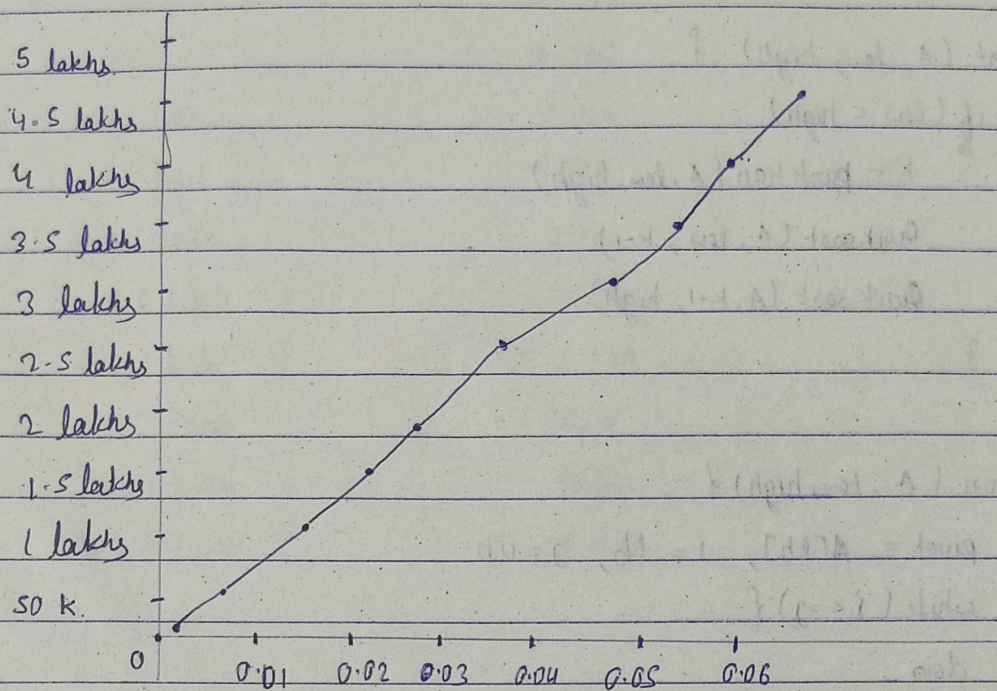
}

Time complexity of Merge sort : $O(n \log n)$

i/p size	Time
100000	0.000
20k	0.002
30k	0.003
40k	0.003
50k	0.003
60k	0.005
70k	0.006
80k	0.007
90k	0.007
1 lakh	0.008

i/p size	Time
10k	0.001
60k	0.006
1.1 lakh	0.013
1.6 lakh	0.019
2.1 lakh	0.024
2.6 lakh	0.031
3.1 lakh	0.045
3.6 lakh	0.047
4.1 lakh	0.052
4.6 lakh	0.055

Graph.



Quick sort Algorithm.

```
QuickSort (A, low, high) {  
    if (low < high)  
        k ← partition (A, low, high)  
        QuickSort (A, low, k-1)  
        QuickSort (A, k+1, high)  
}
```

}

```
partition (A, low, high) {
```

```
    pivot = A[low], i = low, j = high
```

```
    while (i <= j) {
```

```
        do
```

```
            i = j + 1;
```

```
        while
```

```
            pivot >= A[j];
```

```
        do
```

```
            j = j - 1;
```

```
        while
```

```
            pivot < A[i];
```

```
    if (i < j)
```

```
        exchange (A[i], A[j])
```

```
    }
```

```
    exchange (A[j], pivot)
```

```
    return j;
```

```
}
```

i/p size

time

10 k

0.001

60 k

0.006

1.1 lakh

0.014

1.6 lakh

0.026

2.1 lakh

0.041

2.6 lakh

0.054

3.1 lakh

0.079

3.6 lakh

0.11

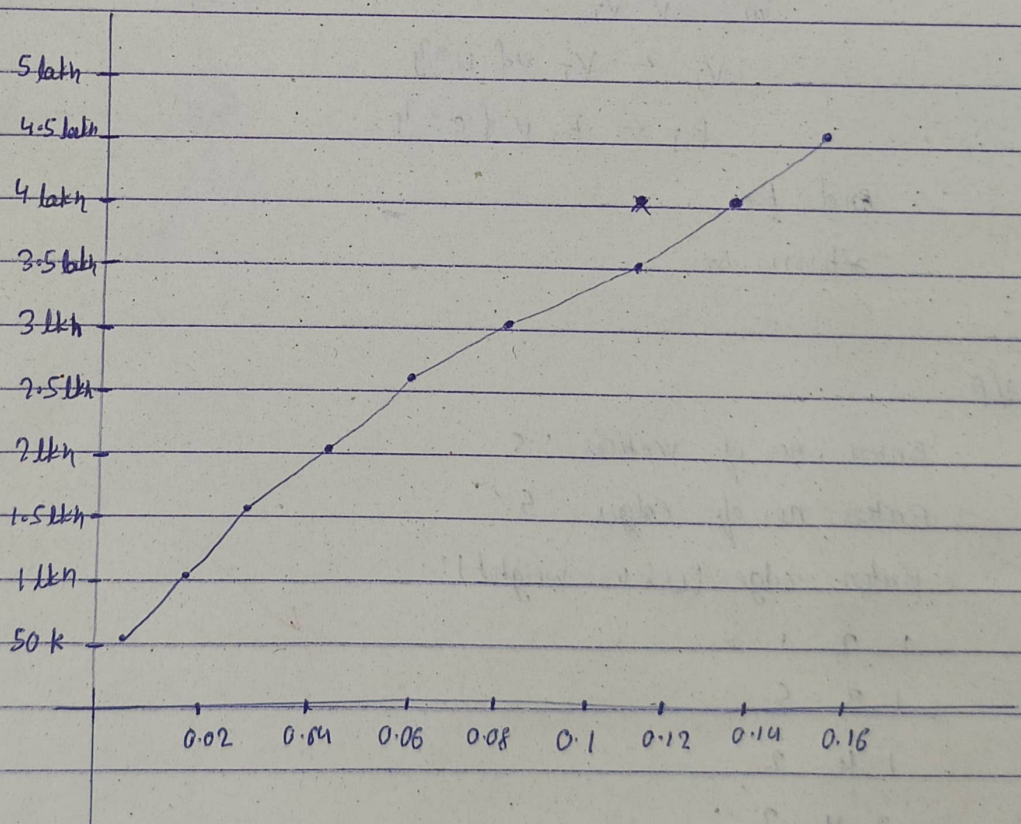
4.1 lakh

0.131

4.6 lakh

0.145

Graph



a) Find the min. cost spanning tree of a given undirected graph using prim's algorithm.

Algorithm for prim's (G)

// Purpose: To construct minimum spanning tree.

// Input: A weighted connected graph.

// Output: The set of edge which form the minimum spanning tree.

$V_T \leftarrow \{V_0\}$, $E_T \leftarrow \emptyset$

for $i \leftarrow 1$ to $|V|-1$ do.

find an edge $e^* = (v^*, u^*)$ with minimum weight among all the edges $e = (v, u)$ such that v^* is in V_T & u^* is in $V - V_T$.

$V_T \leftarrow V_T \cup \{u^*\}$.

$E_T \leftarrow E_T \cup \{e^*\}$.

end for

return E_T

a/p.

Enter no. of vertices: 5

Enter no. of edges: 5.

Enter edge (u, v weight):

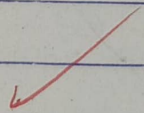
1 2 1

1 3 5.

1 4 2.

3 4 3.

4 5 1.5.



Edge in min. spanning tree.

$$1 \rightarrow 2 = 1$$

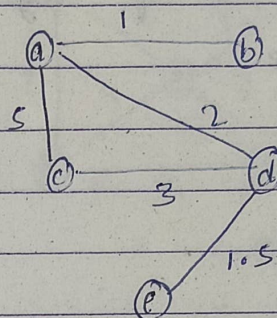
$$1 \rightarrow 4 = 2$$

$$4 \rightarrow 3 = 3$$

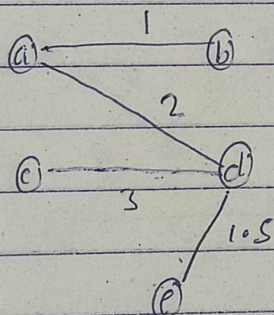
$$4 \rightarrow 5 = 1.5$$

Tot cost of MST = 7.5.

Graph.



MST.



Q) Find min. cost spanning tree of a given undirected graph using Kruskal's algorithm.

Algorithm ^{for} kruskal (G)

// Purpose : To find minimum spanning tree.

// Input : A weighted connected graph G.

// Output : The set of edges that form the minimum spanning tree.

$E_T \leftarrow \emptyset$

count $\leftarrow 0$

$K \leftarrow 0$

while count $< |V|-1$

$K \leftarrow K+1$

if $E_T \cup \{e_{ik}\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_{ik}\}$

count $\leftarrow$ count + 1

end if.

end while

return E_T

a/p

Enter no. of vertices: 5.

Enter no. of edges: 8.

Enter edges (u, v, weight):

1 2 5

1 3 7

1 5 2.

2 5 3.

2 4 6.

3 5 4.

4 3 4.

4 5 5

Edge in min. spanning tree:

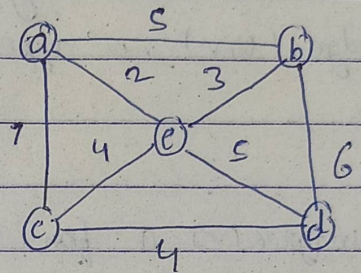
$$1 \rightarrow S = 2$$

$$2 \rightarrow S = 3$$

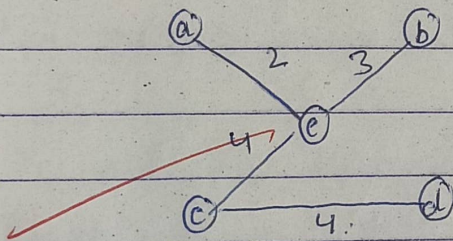
$$3 \rightarrow S = 4$$

$$4 \rightarrow S = 4$$

Total cost of min. MST = 13.



MST



Algorithm for Topological sort (G)

// Purpose: To find the topological ordering of directed acyclic graph

// Input: A directed graph $G = (V, E)$

// Output: A linear ordering of vertices such that for every edge (u, v) , u appears before v

initialize $\text{indegree}[v] = 0$ for all $v \in V$

for each edge $(u, v) \in E$

$\text{indegree}[v] \leftarrow \text{indegree}[v] + 1$

end for

create an empty queue.

for each vertex $v \in V$

if $\text{indegree}[v] = 0$

insert v into Q

end if

end for

count $\leftarrow 0$

TopOrder $\leftarrow$ empty list

While Q is not empty

$u \leftarrow$ delete from Q

add u to TopOrder

count $\leftarrow$ count + 1

0, 1, 2, 3

```

for each vertex v adjacent to v.
    indegree[v] ← indegree[v] - 1
    if indegree[v] == 0
        insert v into Q
    end if
end for
end while.

```

```

if count ≠ |V|
    print "Graph has cycle."
else
    return TopOrder
end if.

```

Algorithm: Topological Sort - source Removal (Graph, n)

// Purpose: to find the topological order of directed acyclic graph
 // Input: Graph with n vertices.
 // Output: Topological ordering of vertices or message if cycle exist.

step 1: Initialize indegree array.

```

for i ← 0 to n-1 do
    indegree[i] ← 0

```

step 2: Compute indegree of each vertex.

```

for i ← 0 to n-1 do
    for j ← 0 to n-1 do
        if Graph[i][j] = 1 then
            indegree[j] ← indegree[j] + 1
        end if
    end for
end for

```

```

end for

```

Step 3: Initialize $count \leftarrow 0$

Step 4: repeat until $count = n$.

found $\leftarrow$ false.

for $i \leftarrow 1$ to $n-1$ do.

if $indegree[i] = 0$ then.

print i

$indegree[i] \leftarrow -1$

$count \leftarrow count + 1$.

found $\leftarrow$ true.

for $j \leftarrow 0$ to $n-1$ do.

if $Graph[i][j] = 1$ then.

$indegree[j] \leftarrow indegree[j] - 1$

end if

end for.

end if

end for.

if found = false. then

print "Graph has a cycle. Topological sort is not possible."
exit.

O/P

Enter number of vertices: 7

Enter number of edges: 7

Enter edges (u v):

0 1

0 2

1 4

1 6

2 5

3 0

3 1

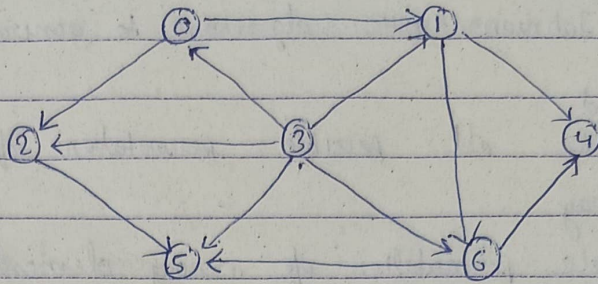
3 2

3 5

3 6

6 5

6 4



Topological Sort (Source removal method):

3 0 1 2 6 4 5

~~Final~~
~~16/4~~

Q) Implement Johnson Trotter algorithm to generate permutation.

I/P: To find all possible permutation of array elements

O/P: An array

I/O: all possible permutation of array elements

Step 1: Initialization.

Assign a direction (left or right) to each element. Initially all element points to left.

Step 2: Find the largest mobile integer.

- An integer is a mobile if it is arrow point to a smaller adjacent integer.

- Among all mobile integers, choose the largest

Step 3: Swap.

Swap the choose mobile integer with adjacent element it points to.

Step 4: Reverse direction

- Reverse the direction of all element greater than the mobile element

Step 5: Repeat until no mobile element exists.

O/P.

Enter no. of elements: 3

Enter the elements: 1 2 3

← 1 ← 2 ← 3

mobile element = 3

← 1 ← 3 ← 2

m.e = 3

← 3 ← 1 ← 2

m.e = 2

→ 3 → 1 ← 2

m.e = 3

(Reverse the direction of 3)

→ 3 → 2 ← 1

m.e = 3

→ 3 → 1 → 2

no mobile element left.

c) Sort element using heap sort bottom-up approach.

• Algorithm

// P : Go sort an unsorted array.

// I : unsorted array

// O : sorted array.

Step 1: Build max heap

~~start from the last non-leaf node & move upward, applying bottom-up heapify~~

Step 2: Bottom-up heapify (sift-down)

• Instead of swapping down step by step

~~heap~~ heap-sort (A, n)

Build-max-heap (A, n)

for $i \leftarrow n-1$ down to 1 do

swap $A[0]$ & $A[i]$

bottom-up-heapify (A, i, 0)

Build-max-heap (A, n):

for $i \leftarrow (n/2 - 1)$ down to 0 do

bottom-up-heapify (A, n, i)

bottom-up-heapify (A, n, i):

temp = $A[i]$

child $\leftarrow 2 * i + 1$

while child $< n$ do

if child + 1 $< n$ & $A[\text{child} + 1] > A[\text{child}]$ then

child $\leftarrow \text{child} + 1$

if $A[child] > temp$ then

$A[i] \leftarrow A[child]$

$i \leftarrow child$

$child \leftarrow 2*i + 1$

else

break

end while

$A[i] \leftarrow temp$

o/p

Enter no. of elements: 4

Enter 4 elements:

13 2 97 1

Original array:

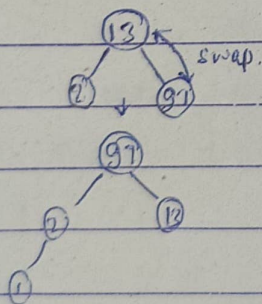
13 2 97 1

Sorted array

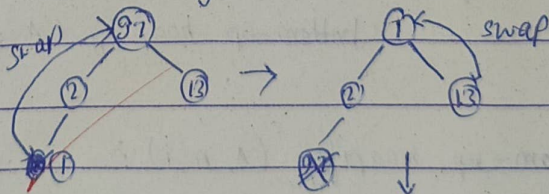
1 2 13 97

Traversing

building max heap

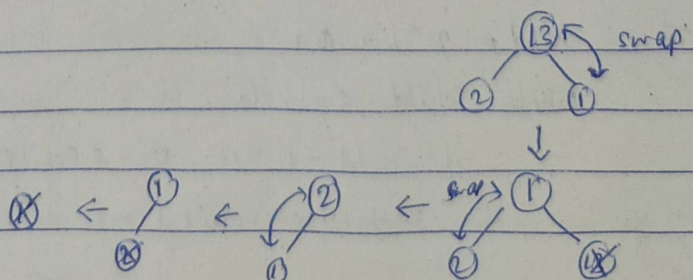


now creating max heap



1 2 13 97

~~13 2 1 97~~
2 1 13 97



leetcode 207.

algorithm:

can-finish (num-courses, prereq)

create adj. list A of size num-course.

int indegree[num-courses]

for each pair (a,b) in prereq:

add a to adj[b]

indegree[a++]

int Q[]

for i ← 0 to num-courses - 1:

if indegree[i] == 0

enq(Q, i)

count ← 0

while Q is not empty:

node ← deq(Q)

count ← count + 1

for each neighbour in A[node]:

indegree[neighbour] --

if indegree[neighbour] == 0:

enq(Q, neighbour)

if count == num courses:

return true

else

false

ex: num courses = 2.

pre req: [[1,0], [0,1]]

dp: false

// cannot complete 0 before 1 while completing 1 before

0.

82.1
27/4

Q) Why Floyd's algorithm.

//P: To find the shortest path b/w every vertices of given graph.

//I: matrix with weights & no. of vertices.

//O: Shortest path from each vertex to another every another vertex.

Algorithm:

~~Flow~~

Floyd's (weighted matrix $[i][j], n$)

for $i \leftarrow 0$ to $n-1$ do.

for $j \leftarrow 0$ to $n-1$ do

for $k \leftarrow 0$ to $n-1$ do.

if $((dis[i][k] + dis[k][j]) < dis[i][j])$

$dis[i][j] = dis[i][k] + dis[k][j]$

end if.

end for

end for

end for.

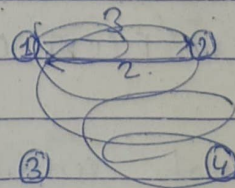
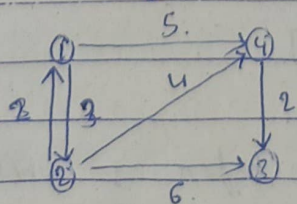
O/P

Enter no. of vertices : 4

Enter weighted adjacency matrix :

(Enter 99999 if no edge exists)

0	3	99999	5
2	0	99999	4
99999	6	0	99999
99999	99999	2	0



Shortest distance matrix

0 3 7 5

2 0 6 4

8 6 0 10

10 8 2 0

Increasing

$$D^0 = \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 6 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix}$$

→

$$D^1 = \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 6 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix}$$

↓

$$D^3 = \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ 8 & 6 & 0 & 10 \\ 10 & 8 & 2 & 0 \end{bmatrix}$$

←

$$D^2 = \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ 8 & 6 & 0 & 10 \\ \infty & \infty & 2 & 0 \end{bmatrix}$$

↓

~~$$D^4 = \begin{bmatrix} 0 & 2 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 8 & 6 & 0 & 10 \\ 10 & 8 & 2 & 0 \end{bmatrix}$$~~

Ques 1334

```
#define INF 1000000
```

```
int findTheCity(int n, int** edges, int edgesize, int* edgeColSize, int distance -  
threshold)
```

```
int dist[n][n];
```

```
for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i == j)
```

```
            dist[i][j] = 0;
```

```
        else
```

```
            dist[i][j] = INF;
```

```
    }
```

```
}
```

```
for (int i = 0; i < edgesize; i++) {
```

```
    int u = edges[i][0];
```

```
    int v = edges[i][1];
```

```
    int w = edges[i][2];
```

```
    dist[u][v] = w;
```

```
    dist[v][u] = w;
```

```
}
```

```
for (int k = 0; k < n; k++) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
```

```
                dist[i][j] = dist[i][k] + dist[k][j];
```

```
            }
```

```
        }
```

```
    }
```

```
int answer = -1;
```

```
int minCount = INF;
```

```
for (int i = 0; i < n; i++) {
```

```
    int count = 0;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (dist[i][j] <= distanceThreshold) {
```

```
            count++;
```

```
        }
```

```
    }
```

```
    if (count <= minCount) {
```

```
        minCount = count;
```

```
        answer = i;
```

```
    }
```

```
}
```

```
return answer;
```

```
}
```

d/p.

n = 4.

edges = [[0,1,3], [1,2,1], [1,3,4], [2,3,1]]

distanceThreshold = 4

d/p 3.

~~ans = 1~~
~~ans = 5/26~~

Dijkstra's algorithm $(G, s) \rightarrow T.C.$

// P:

// i/p:

// o/p:

for $i \leftarrow 1$ to n do

$dist[i] = \infty$

$visited[i] = false$

end for

$dist[s] \leftarrow 0$

for $i \leftarrow 1$ to n do

$u \leftarrow$ vertex with min $dist[u]$ among unvisited vertices

$visited[u] = true$

 for each vertex v adjacent to u do

 if $visited[v] = false$ & $dist[u] + w(u, v) < dist[v]$ then

$dist[v] = dist[u] + w(u, v)$

 end if

 end for

end for

o/p

Enter number of vertices: 5.

Enter adjacency matrix

0 3 0 7 0

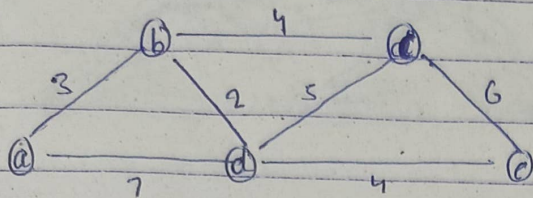
3 0 4 2 0

0 4 0 5 6

7 2 5 0 4

0 0 6 4 0

Enter source vertex: 0.



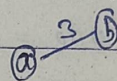
tree vertices

remaining vertices

illustration

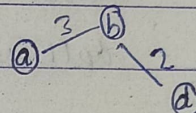
$a(-, 0)$

$\checkmark$
 $b(a, 3), d(a, 7)$



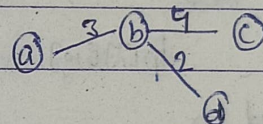
$b(a, 3)$

$\checkmark$
 $d(b, 5), c(b, 7)$



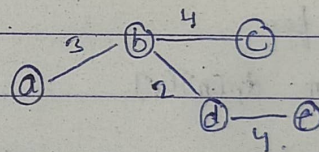
$d(b, 5)$

$\checkmark$
 $c(d, 10), e(d, 9), c(b, 7)$



$c(b, 7)$

$\checkmark$
 $e(c, 13), e(d, 9)$



01 knapsack algorithm (weight, values, n, w)

create $dp[n+1][w+1]$

for $i \leftarrow 0$ to n

for $w \leftarrow 0$ to w

if $i=0$ or $w=0$

$dp[i][w] = 0$

else if $(weight[i-1] \leq w)$

include = $value[i-1] + dp[i-1][w - weight[i-1]]$

exclude = $dp[i-1][w]$

$dp[i][w] = \max(include, exclude)$

else

$dp[i][w] = dp[i-1][w]$

end for

end for

return $dp[n][w]$

a/p.

Enter number of items: 4

Enter values of items:

12 10 20 15

Enter weight of items:

2 1 3 2

Enter capacity of knapsack: 5

Maximum profit = 37

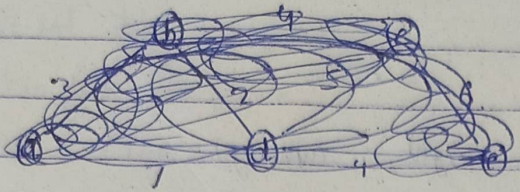
Selected items:

Item $\rightarrow$ 4, value = 15, weight = 2, Quantity = 1

Item $\rightarrow$ 2, value = 10, weight = 1, Quantity = 1

Item $\rightarrow$ 1, value = 12, weight = 2, Quantity = 1

total weight used = 5



leetcode - 494

```
int findTargetSumWays (int * nums, int numsSize, int target) {
    int totalSum = 0;

    for (int i = 0; i < numsSize; i++) {
        totalSum += nums[i];
    }

    if ((target + totalSum) % 2 != 0 || totalSum < abs(target)) {
        return 0;
    }

    int subsetSum = (target + totalSum) / 2;
    int dp[subsetSum + 1];

    for (int i = 0; i <= subsetSum; i++) {
        dp[i] = 0;
    }

    dp[0] = 1;

    for (int i = 0; i < numsSize; i++) {
        for (int j = subsetSum; j >= nums[i]; j--) {
            dp[j] += dp[j - nums[i]];
        }
    }

    return dp[subsetSum];
}
```

O/p

nums = [1, 1, 1, 1, 1]

target = 3

O/p = 5

Fractional Knapsack

Algorithm fractional-knapsack (profit[], weight, n, capacity)

create items with:

profit, weight, ratio = profit / weight

sort items in descending order of ratio.

total profit = 0

for each item in sorted items.

if capacity $\geq$ item.weight.

take full item.

total profit = total profit + item.profit

capacity = capacity - item.weight

else.

fraction = capacity / item.weight.

total profit = total profit + (item.profit * fraction)

break.

end for

return total profit.

c/p.

Enter number of items: 5.

Enter profit and weight of item 1: 20 10

Enter profit and weight of item 2: 30 20.

Enter profit and weight of item 3: 66 30

Enter profit and weight of item 4: 40 40

Enter profit and weight of item 5: 60 50.

Enter capacity: 100

Selected items:

Item 1 $\rightarrow$ 100% taken

Item 2 $\rightarrow$ 100% taken

Item 3 $\rightarrow$ 100% taken

Item 4 $\rightarrow$ 86.00% taken

maximum profit = 164.00

leetcode - 316

char* removeDuplicateLetters (char* s) {

int last[26] = {0};

int visited[26] = {0};

int n = strlen(s);

for (int i = 0; i < n; i++) {

last[s[i] - 'a'] = i;

}

char* stack = (char*) malloc((n+1) * sizeof(char));

int top = -1;

for (int i = 0; i < n; i++) {

char c = s[i];

if (visited[c - 'a'])

continue;

while (top >= 0 && stack[top] > c && last[stack[top] - 'a'] > i) {

visited[stack[top] - 'a'] = 0;

top--;

}

stack[++top] = c;

visited[c - 'a'] = 1;

}

stack[top+1] = '\0';

return stack;

}

o/p

input s = "bcabc"

o/p "abc"

✓ ~~224~~ 1115

N-Queen's problem

Algorithm for n-queen's

// P: find possible no. of ways in which n-queen's can be arranged.

// I/P: no. of queen's

// O/P: matrix showing possibility.

input n

create chessboard[n][n] & initialise all cells to 0

func: isSafe(board, row, col, n)

check left side of current row

if queen exists

return false

check upper-left diagonal

if queen exists

return false

check lower-left diagonal

if queen exists

return false

return true

end func.

func: solve(board, col, n)

if col >= n

return true

for each row from 0 to $n-1$

if isSafe(board, row, col, n)

place queen at board[row][col]

if solve(board, col+1, n) == true

return true

remove queen from board[row][col]

end for

return false

end func

call solve(board, 0, n)

if solution exists

print board

else

print "Solution does not exist"

end

o/p

Enter no. of queens: 4

Solution 1:

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Solution 2:

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Total solutions = 2

~~25/05/26~~